

# C++ Compile

Rene Rivera – grafikrobot@gmail.com – P1178R0, 2018-10-06 | Initial

---

## Table of Contents

- 1. Introduction
  - 1.1. Background and Motivation
  - 1.2. Impact On the Standard
  - 1.3. Design Goals
  - 1.4. Design Decisions
    - 1.4.1. Exceptions
    - 1.4.2. Argument Exclusivity
    - 1.4.3. + Option Prefix
    - 1.4.4. Use Cases
  - 1.5. About Linking
  - 1.6. Link Compatibility
- 2. Argument Specifications
  - 2.1. Core Arguments
  - 2.2. Experimental Arguments
  - 2.3. Vendor Arguments
- 3. Future Work
- 4. Technical Specification
  - 4.1. Header `<compile>` Synopsis
  - 4.2. Specification
    - 4.2.1. Arguments
      - 4.2.1.1. Long & Short Options
      - 4.2.1.2. Core Options
      - 4.2.1.3. Experimental Options
      - 4.2.1.4. Vendor Options
      - 4.2.1.5. Compatibility
      - 4.2.1.6. Options
- 5. Prior Art
  - 5.1. `clang-cl`
  - 5.2. `gxlcpp`
  - 5.3. NVRTC
  - 5.4. `B2` build system
  - 5.5. Cmake build system
- 6. Acknowledgements

---

|                        |                                    |
|------------------------|------------------------------------|
| <b>Document number</b> | ISO/IEC/JTC1/SC22/WG21/P1178R0     |
| <b>Date</b>            | 2018-10-06                         |
| <b>Reply-to</b>        | Rene Rivera, grafikrobot@gmail.com |
| <b>Audience</b>        | Tooling (SG15), Library Evolution  |

# 1. Introduction

This proposes to add an interface for transforming C++ source into usable executable programs. As such it aims to provide a common definition of compiler frontend tool arguments and options for transforming source code to translation units and linking those into executable programs.

## 1.1. Background and Motivation

This proposal is based on the notion that it should be possible for a C++ user, both beginner and advanced, to easily transform their source to immediately executable form in a consistent method across compilers.

One aspect of this proposal is to make it possible for future C++ educators, and book authors, to provide complete usage examples that work for any particular tool they and their audience use. While this proposal does not prescribe a particular executable tool, it is trivial to write minimal tool that passes arguments directly to these interfaces. It is hoped that vendors would provide ready made compiler front ends for their tools accepting the standard arguments.

It is also the intent for this proposal to be an initial step in the road towards standardizing the tooling around C++. With the goal of having a global and consistent tooling ecosystem to facilitate the introduction into C++ for anyone and to ease the interoperability of future tooling like package and dependency managers by having a single common set of options to base solutions on.

Additionally this facility could also support runtime compilation, for example for runtime generated source, and other compilation models where the particular execution platform is not known ahead of time.

## 1.2. Impact On the Standard

This proposal is a library extension and specification. As such it does not prescribe changes to any standard classes or functions. It adds one function, and overloads, in a new header file, `<compile>`.

## 1.3. Design Goals

The design herein is driven primarily by the following, in approximate order of importance:

- Mirror the `main` entry point to make a frontend program trivial.
- Minimize variability in arguments and options.
- Use common practice option names for familiarity.
- Do not attempt to specify all possible options as that's not technically possible.
- Specify how, and if, options are compatible with others to facilitate tooling checks for binary compatibility.

## 1.4. Design Decisions

### 1.4.1. Exceptions

So as to reduce error handling cases for callers of `compile`, i.e. `main`, there are no exceptions allowed from calls to `compile`. This has the effect of removing the need for exception handling at the caller as any error situations are indicated by the simpler return value which can be directly mapped to the return value of `main`.

### 1.4.2. Argument Exclusivity

Recognizing that specifying a complete set of argument options is not possible we need to provide for a mechanism to pass vendor specific arguments for compilation; but at the same time we want to ensure that defined arguments are not ignored in favor of vendor only arguments. Hence we specify this implementation rule:

When a defined core argument option exists an equivalent vendor specific argument is **not** allowed as an acceptable argument option.

#### For instance

We specify the `++address_model` core option. It would not be allowed for a vendor to specify an `++v:xyz:m64` (to specify 64 bit address model) option as that would overlap with the core option.

#### 1.4.3. + Option Prefix

Like choosing names for programming language constructs, choosing a prefix for the option arguments is a contentious decision. For this we chose to use `+` as the prefix, and hence `++` for the long form, for these reasons:

- It's not used widely by current compiler tooling and hence avoids confusion with existing options.
- It doesn't conflict with operating system specific path specifications.
- Is a non-alphanumeric ASCII glyph.
- Not frequently used as the first character of file names.
- And finally, it harkens to C++ itself.

Regardless of this choice, this is ostensibly a straw-man choice. There are valid reasons to choose other prefix (say `-`). And this proposal does not hinge on what this prefix is. But do note that the sample implementation does use `+` and `++`.



There is at least one class of compilers that historically use the `+` prefix. For example the Verilog-XL ([https://www.csee.umbc.edu/portal/help/VHDL/verilog/command\\_line\\_plus\\_options.html](https://www.csee.umbc.edu/portal/help/VHDL/verilog/command_line_plus_options.html)) uses the `+` option prefix exclusive.

#### 1.4.4. Use Cases

This facility is designed to facilitate primarily these groups of users: teachers (and their students), and tool creators. For teachers this feature would improve the teaching materials presented to students.

#### For instance

This is the first code example from a popular C++ programming book:

“For example:

```
#include<iostream>

int main()
{
    std::cout << "Hello, new world!\n";
}
```

C++

Which is shortly followed by this explanation:

“The language used in this book is “pure C++” as defined in the C++ standard [C++,1998]. Therefore, the example ought to run on every C++ implementation. The major program fragments in this book were tried using several C++ implementations.

But how to run and compile examples is not described in the entirety of the book for good reasons. As it would take a whole chapter onto itself just to skim at that subject given the current state of compiler tools. With this proposal it would at least be possible to give a short explanation of what a compiler invocation would look like. The above example could be followed by:

“You can compile the example with this command, substituting a compiler program of your choice:

```
> cpp hello.cpp ++output=hello.exe
```

And run it by invoking the resulting program:

```
> hello.exe  
Hello, new world!
```

For tool creators the uses vary depending on the tools in question. They could be a build system, like `b2` (aka Boost.Build) that has complicated toolset definition files that translate portable build descriptions to specific tool invocations.

#### For instance

In `b2` one would indicate include search paths by adding `<include>my/include` to the specification of the build definition. It will then translate that to a compile invocation depending on the tool: `cc -Imy/include, bcc32.exe -I"my/include", clang -I"my/include", gcc -I"my/include", cl.exe /I"my/include"`, and so on. This is obviously a very simple example and the options get less uniform across compilers the more "esoteric" the options are. And the build system needs lots of human programmed knowledge to create these mappings. Instead with this proposal the mapping would be universal and only non-core options and the tool executable would need special treatment.

Publishing libraries, with or without source, also poses a problem for those doing the publishing. As part of publishing, your library will have specific options it needs a program to compile with to make use of the library. This can involve rather complex distributions to account for the varied compilers end users have. And many times this involves publishing many different files for each possible configuration and compiler you support.

#### For instance

One common method of doing that is to use `pkg-config`. You might end up with this kind of file for GCC:

```
Name: libpng
Description: Loads and saves PNG files
Version: 1.2.8
Libs: -L${libdir} -lpng12 -lz
Cflags: -I${includedir}/libpng12
```

And another for MSVC:

```
Name: libpng
Description: Loads and saves PNG files
Version: 1.2.8
Libs: /LIBPATH:${libdir} png12.lib z.lib
Cflags: /I${includedir}/libpng12
```

With this proposal it would be possible to only include one such file regardless of compiler used:

```
Name: libpng
Description: Loads and saves PNG files
Version: 1.2.8
Libs: ++library_path=${libdir} +lpng12 +lz
Cflags: +I${includedir}/libpng12
```

The uses are not limited to the above two groups of users though. This facility also opens the door for some users that need to process generated source into executable form.

### For instance

One can have a system that uses database schema definitions to generate C++ source code to implement that schema, and related queries, in an optimal form backed by a custom C++ database engine. This would let one do that operation in a much simpler and direct method.

## 1.5. About Linking

When considering this extension the question comes up about the definition of generating compiled source files and linking them into a program. The key issue is that there is the claim that there is no normative text in the standard that defines the process of separate compilation and linking.

My research, with regards to reading the standard text, is that even though the standard does tread carefully to avoid specifics about the interaction of external linkage, compiling, and the resulting program; There is a definite understood specification that one "translates" source code units into separate "blobs" that are then "linked" into the final runnable program. Specifically section 6.5 ([basic.link]) defines this process with:

*A program* consists of one or more *translation units* (Clause 5) linked together. A translation unit consists of a sequence of declarations.

This proposal doesn't claim to define the compilation and linkage any further than is currently defined. The options mandated herein do not specify any particular process or mechanisms past the existing specification of program and linkage. This proposal does not:

1. Prescribe any particular storage mechanism for source code, translated source code, nor a program.

2. Prescribe any particular format for translated source code nor program.

And hence the specifics of what the results of compiling the translation units into linkable outputs and linking those outputs into an executable program is entirely implementation defined.

## 1.6. Link Compatibility

This proposal defines a *link compatibility* for options to make it clear what compiled translation units are compatible with each other. This information facilitates various use cases:

- It makes it possible for build systems to detect and respond to the compatibility. To either notify the user of the problem, or to create variations that are compatible with your request.
- It makes it possible for package managers that support pre-compiled binary libraries to detect if the available packages are usable with your program. And possibly adjust to find the particular binary variation that will work for your case.

## 2. Argument Specifications

### 2.1. Core Arguments

Core arguments encompass those that are globally available in all compilers. Their specification needs to be done through a mechanism that requires compiler vendors to implement them to be able to attain a minimal usable API. At this time the available method for achieving this is to include the specification of core arguments in the *International Standard*. Which is what we propose herein with the `std::compile` function.

#### Requirements for core arguments

- Implementable by many compilers.
- Already in wide use.
- Experiential knowledge of behavior as an *experimental argument* or *vendor argument*.
- Have a meaningful, but terse, long form name.

### 2.2. Experimental Arguments

Experimental arguments allow for a period of review before adding core arguments. We recognize that it's important to acquire experience with arguments before adding them to the International Standard. We propose to have a standing document defining arguments that are intended to become core arguments but that otherwise do not have sufficient real-world implementation experience. Benefits include:

- Allowing time for tool implementations to support the change.
- Acquire experience on compiler, and other tools, support.

Recognizing that experimental arguments will most likely come from vendor arguments, and that there needs to be a period of for tools to adjust, vendor arguments would be allowed to overlap with experimental arguments, unlike core arguments.

### 2.3. Vendor Arguments

Given the faster rate at which compilers change with respect to the C++ standard we need to allow for specification of arguments as quickly as possible to facilitate support in tooling.

**For instance**

A vendor releases a new version of their compiler to support a new CPU. This new CPU is not compatible with an existing CPU and hence packages must be recompiled for it. But the next scheduled committee meeting that could approve adding that new CPU option is months later.

Hence we propose the use *Standing Documents* for specifying the vendor specific options and for the highly volatile option values as needed. Such that SG15 could meet and approve changes to those *Standing Documents* as needed in a timely matter.

### 3. Future Work

This is currently a limited and incomplete technical specification. Future revisions will grow the set of core and vendor options. The current options were chosen as a minimal working set to create working programs with minimal compilation complexity.

There are various existing tools and APIs that either perform argument mapping from one compiler to another, or are direct compiler frontend drivers. There will be a more thorough listing of such tools and APIs in future revisions.

We will also add background as to which C++ tools benefit from standard argument options, and how, in future revisions.

### 4. Technical Specification



This is a **very rough** draft of a specification. As such it will contain errors and not be written in the most precise manner.

#### 4.1. Header `<compile>` Synopsis

```
namespace std
{
    int compile(int, char * *) noexcept;
    int compile(vector<string>) noexcept;
}
```

#### 4.2. Specification

```
int compile(int argument_count, char * * arguments) noexcept;
```

**Requires** `argument_count > 0`, where `arguments[0]` is the name of the program executable. And `arguments[1..argument_count)` are either options or names of source files.

**Effect** Executes the compilation according to the given `arguments`. Implementations should indicate an error if the functionality is not available.

**Returns** zero (0) for success, non-zero (`!= 0`) for an error.

```
int compile(vector<string> arguments) noexcept;
```

**Requires** `!arguments.empty()`, where `arguments[0]` is the name of the program executable. And `arguments[1..N)` are either options or names of source files.

**Effect** Executes the compilation according to the given `arguments`. Implementations should indicate an error if the functionality is not available.

**Returns** zero ( 0 ) for success, non-zero ( != 0 ) for an error.

#### 4.2.1. Arguments

The `arguments` specify the set of options, translation units to transform, or transformed translation units to link into a program. An argument is either an option and option value, or a name of a translation unit source. Options start with the `+` or `++` option prefix.

##### 4.2.1.1. Long & Short Options

Arguments that start with `++` are *long options*. And arguments that start with a single `+` are *short options*. *Long options* are expected to be succinct but reasonably human readable names of options. And *short options* are expected to be well known one or two character alternatives to the *long options*.

##### 4.2.1.2. Core Options

*Core options* follow this syntax:

```
++name[=value]
++name [value]
+n [value]
```

Where the *value* for the option may be optional, and may have additional syntax constraints, as specified for any particular option.

##### 4.2.1.3. Experimental Options

*Experimental options* follow the same syntax as *core options* and are specified in a *standing document*.

##### 4.2.1.4. Vendor Options

*Vendor options* are specified in a vendor specific *standing document* and follow this syntax:

```
++vendor-key:name[=value]
++vendor-key:name [value]
+vendor-key:n [value]
```

Where the *value* for the option may be optional, and may have additional syntax constraints, as specified for any particular option. And where *vendor-key* is a unique, short, name for the vendor specific compiler.

##### 4.2.1.5. Compatibility

Options, and their values, can have a *compatibility* of:

###### ***Link incompatible***

Wherein translation units transformed with differing options or values **can not** be linked together into a program.

###### ***Link compatible***

Wherein translation units transformed with differing options or values **can** be linked together into a program.

###### ***Implementation defined***



Wherein translation units transformed with differing options or values are compatible depending on the particular C++ implementation.

## NA

Wherein the compatibility is not directly impacted by the option or values.

Options that are *link incompatible* with other options also indicate which other options affect compatibility. Otherwise a *link incompatible* option is only incompatible between its own different values.

### 4.2.1.6. Options

#### 4.2.1.6.1. help

```
+h
+?
++help
```

**Effect** Displays all available options and exits.

#### 4.2.1.6.2. output

```
+o file
++output=file
```

**Argument** An implementation defined path to a *file*.  
**Effect** Sets the destination file of the compilation.  
**Support** Required  
**Compatibility** NA

#### 4.2.1.6.3. define

```
+D definition
++define=definition
```

**Argument** A preprocessor *definition* of the form *name=value*  
**Effect** Instructs the preprocessing phase to define the given *name* symbol to the given *value*, or to `1` if *value* is not given.  
**Support** Required  
**Compatibility** NA

#### 4.2.1.6.4. include directory

```
+I directory
++include-dir=directory
```

**Argument** An implementation defined path to a *directory*.  
**Effect** Adds the given *directory* to the end of the search path of the preprocessor.  
**Support** Required  
**Compatibility** NA

#### 4.2.1.6.5. debug information

```
+g
++debug
```

|                      |                                                             |
|----------------------|-------------------------------------------------------------|
| <b>Argument</b>      | None                                                        |
| <b>Effect</b>        | Adds source-level debug information to the compiled output. |
| <b>Support</b>       | Required if supported by the environment.                   |
| <b>Compatibility</b> | NA                                                          |

#### 4.2.1.6.6. standard conformance

```
+std standard
++standard=standard
```

|                      |                                                                                                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Argument</b>      | One of: <code>98</code> , <code>03</code> , <code>11</code> , <code>14</code> , <code>17</code> , or <code>2a</code> indicating the year of the <i>standard</i> . |
| <b>Effect</b>        | Instructs compilation to use the given <i>standard</i> as the version of the C++ language standard for the source TU.                                             |
| <b>Support</b>       | The language levels accepted, within the allowed values, is implementation defined.                                                                               |
| <b>Compatibility</b> | Link incompatible.                                                                                                                                                |

#### 4.2.1.6.7. warnings

```
+W warning_option
++warnings=warning_option
```

|                      |                                                                                                                                                                                                                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Argument</b>      | One of: <code>off</code> , <code>on</code> , <code>all</code> , <code>error</code> indicating the level of warnings to report and whether to treat warnings as errors.                                                                                                                                                    |
| <b>Effect</b>        | Instructs compilation to: not report warnings ( <code>off</code> ), report a default set of warnings that is implementation defined ( <code>on</code> ), to report all possible warnings ( <code>all</code> ), and if instead of reporting warnings they should be considered compilation errors ( <code>errors</code> ). |
| <b>Support</b>       | Implementations are only required to have an effect for <code>off</code> and <code>on</code> values. Other unimplemented values must be ignored.                                                                                                                                                                          |
| <b>Compatibility</b> | Link compatible.                                                                                                                                                                                                                                                                                                          |

#### 4.2.1.6.8. optimize

```
+O optimization_level
++optimize=optimization_level
```

|                      |                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Argument</b>      | One of: <code>off</code> , <code>on</code> , <code>speed</code> , or <code>size</code> .                                                         |
| <b>Effect</b>        | Instructs compilation to optimize the generated code using the indicated level and type.                                                         |
| <b>Support</b>       | Implementations are only required to have an effect for <code>off</code> and <code>on</code> values. Other unimplemented values must be ignored. |
| <b>Compatibility</b> | Link compatible.                                                                                                                                 |

#### 4.2.1.6.9. address\_model

```
++address_model=bits
```

|                      |                                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Argument</b>      | A positive number greater than 0 indicating the number of bits to base pointer and number generated instructions on.       |
| <b>Effect</b>        | Instructs compilation to generate instructions that conform to an implementation defined size of pointers and numbers.     |
| <b>Support</b>       | The values accepted are implementation defined. And any not understood values shall fail compilation and produce an error. |
| <b>Compatibility</b> | Link incompatible.                                                                                                         |

#### 4.2.1.6.10. *library*

```
+l library  
++library=library
```

|                      |                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Argument</b>      | An implementation defined name of the <i>library</i> .                                                                      |
| <b>Effect</b>        | Adds the given <i>library</i> to the program linking as an already compiled single translation unit, or collection thereof. |
| <b>Support</b>       | Required                                                                                                                    |
| <b>Compatibility</b> | NA                                                                                                                          |

#### 4.2.1.6.11. *library directory*

```
+L directory  
++library-dir=directory
```

|                      |                                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Vendor</b>        | gcc                                                                                                                               |
| <b>Argument</b>      | An implementation defined path to a <i>directory</i> .                                                                            |
| <b>Effect</b>        | Adds the given <i>directory</i> to the end of the search path for finding libraries specified with <code>++library</code> option. |
| <b>Compatibility</b> | NA                                                                                                                                |

## 5. Prior Art

### 5.1. `clang-cl`

The Microsoft Visual Studio development environment supports using Clang as an alternative compiler to their `msvc` (`cl.exe`) compiler. In order to avoid reflecting the Clang specific options up into the IDE `clang-cl` implements a compiler front end that translates from `msvc` options to Clang options.

`clang-cl` (<http://clang.llvm.org/docs/UsersManual.html#clang-cl>) is an alternative command-line interface to Clang, designed for compatibility with the Visual C++ compiler; `cl.exe`.

## 5.2. gxlcpp

The IBM XL C/C++ compiler implements a translation driver program that provides a GCC compatible frontend. The express goal of the utility is to "help you reuse make files created for applications previously developed with GNU C/C++".

XL C and XL C/C++ provides the `gxlcc` and `gxlcpp` (<https://www-01.ibm.com/support/docview>) utilities to map many GCC compiler options to their XL C and XL C/C++ counterparts.

## 5.3. NVRTC

There is an existing example of providing a direct interface to the compiler:

`NVRTC` (<https://docs.nvidia.com/cuda/nvrtc/index.html>) is a runtime compilation library for CUDA++

It provides for a way to take CUDA specific C++ source (as data) and compile it to produce a CUDA specific binary executable program. Although not as simple of an interface as we are proposing in this extension it's very similar in approach. Providing for the arguments to control the compile as `options` :

```
nvrtcResult nvrtcCompileProgram ( nvrtcProgram prog, int numOptions, const char** options )
```

## 5.4. B2 build system

The `B2` (aka Boost Build) build system uses the concept of features to portably abstract the declaration of compiler options to a portable specification. For example:

- `optimization=off` would add the compiler specific option to turn off any code optimization.
- `debug-symbols=on` would add the compiler specific option to add source level debugging information to the output.
- `address-model=64` would add compiler specific options to generate 64-bit addressing only code.

## 5.5. Cmake build system

Like the features of `b2` modern Cmake also implements a series of variables that provide an abstraction to common compiler options. For example:

- `set_property(TARGET foo PROPERTY CXX_STANDARD 11)` would add the compiler specific option to restrict compilation to use Standard C++11 features.

# 6. Acknowledgements

Thanks to CppCon 2017 for providing the environment for "arguments" about build systems, package managers, and dependency managers that created the impetus for this idea.

Thanks to Jonathan Müller, Ben Craig, Corentin Jabot, Dan Kalowsky, Steve Downey, Breno Rodrigues Guimarães, Robert Maynard, and others in the CppLang Slack community who provided feedback to the draft version of this document.

Thanks to JF Bastien, Peter Sommerlad and others in the SG15 reflector for providing valuable insights from the eastrly draft of this document.

Special thanks to Peter Dimov and Vinnie Falco for reading and pointing out some obvious English spelling and grammar mistakes.